

第12讲 性能优化、团队协作与AI深度应用

一、课程基本信息

项目	内容
课程名称	移动应用开发
授课章节	第六章 综合开发实践（第2讲）
授课学时	2学时
授课方式	理论讲授 + 案例分析
授课教师	刘东良
适用专业	软件工程

二、教学目标

知识目标

- 掌握移动应用性能优化的核心维度与常用方法
- 理解移动应用发布流程与应用商店上架规范
- 掌握代码质量保障的方法与工具链
- 理解Git分支管理策略与代码审查流程
- 了解AI工具在移动开发中的深度应用场景
- 理解工匠精神与团队协作的职业素养内涵

能力目标

- 能够对移动应用进行性能分析与优化
- 能够制定应用发布计划并处理上架问题
- 能够运用代码规范与静态分析工具保障代码质量
- 能够运用Git进行团队协作与分支管理
- 能够使用AI工具辅助代码重构、性能优化与测试生成

6. 具备团队协作与项目迭代的综合实践能力

素质目标

1. 培养精益求精的工匠精神与质量意识
2. 树立团队协作精神与沟通协调能力
3. 增强持续学习与技术创新意识
4. 培养职业道德与责任感

三、教学重点与难点

教学重点

1. 启动速度优化、内存管理与泄漏检测
2. 网络请求优化与UI渲染性能调优
3. 代码规范与静态分析工具使用
4. Git分支管理策略与代码审查
5. AI工具在代码重构与测试生成中的应用

教学难点

1. 性能瓶颈的定位与分析方法
2. 内存泄漏的检测与修复
3. 不同团队规模的分支策略选择
4. AI工具应用的边界与质量把控
5. 工匠精神在项目迭代中的落地实践

四、教学内容与过程设计

第一学时：性能优化与应用发布（45分钟）

1. 课程导入（5分钟）

- **数据导入**：应用性能与用户留存率的关系（每慢1秒，流失率增加7%-20%）
- **案例分析**：某热门APP的性能优化历程与效果对比

- **学习目标：**掌握性能优化方法论，打造高质量移动应用
- **知识地图：**性能优化 → 应用发布 → 代码质量 → 团队协作 → AI应用

2. 性能优化全景 (20分钟)

为什么要做性能优化？

- 用户体验：流畅的应用带来更高的满意度
- 业务指标：更快的加载=更高的转化率
- 设备适配：中低端设备也能流畅运行
- 口碑传播：好的性能带来好的评价

性能优化五大维度

维度一：启动速度优化

- 启动类型：冷启动、温启动、热启动
- 冷启动优化策略：
 - Application.onCreate 减负：延迟初始化、异步初始化
- 启动页优化：精简布局、避免过度绘制
- 懒加载：非核心功能延后加载
- 类预加载：合理使用启动期预加载
- 启动时间测量：
 - Android: Logcat、Systrace、Perfetto
 - iOS: Xcode Instruments、Time Profiler
- 优化目标：冷启动 < 2秒为优秀

维度二：内存管理与泄漏检测

- 内存管理基础：
 - Java/Kotlin: GC (垃圾回收) 机制
 - Swift: ARC (自动引用计数)
 - Dart: GC + 引用计数
- 常见内存泄漏场景：
 - 静态变量持有Activity/Context引用
 - 非静态内部类持有外部类引用 (Handler、Thread)
 - 监听器未注销
 - 单例持有Context
 - 资源未关闭 (Cursor、File、Stream)
- 内存检测工具：
 - Android: LeakCanary、Android Studio Profiler、MAT
 - iOS: Instruments Leaks、MLeaksFinder

- Flutter: DevTools
- 内存优化策略:
- 合理使用缓存, 设置上限
- 图片压缩与采样
- 避免创建过多临时对象
- 大对象复用 (对象池)

维度三: 网络请求优化

- 网络性能指标: 请求耗时、成功率、流量消耗
- 优化策略:
- 缓存策略: HttpCache、本地缓存
- 请求合并: 减少请求次数
- 数据压缩: Gzip、Protocol Buffer
- 图片优化: WebP格式、按需加载、缩略图
- 预加载: 预判用户行为提前加载
- 弱网优化: 降级处理、重试机制
- DNS优化: HTTPDNS、预解析
- 网络监控: 抓包工具 (Charles、Fiddler) 、APM工具

维度四: UI渲染性能调优

- 渲染原理: CPU (布局计算) → GPU (绘制) → 显示
- 60fps与16ms原则: 每帧必须在16ms内完成
- 常见性能问题:
- 过度绘制 (Overdraw) : 背景叠加
- 布局层级过深: measure/layout耗时
- 频繁重绘: onDraw中创建对象
- 列表卡顿: ViewHolder模式、异步加载
- 检测工具:
- Android: GPU过度绘制调试、Hierarchy Viewer、Systrace
- iOS: Core Animation工具、Color Blended Layers
- 优化方法:
- 减少布局层级: 使用ConstraintLayout、扁平化布局
- 避免过度绘制: 移除不必要的背景
- 列表优化: ViewHolder复用、分页加载
- 异步布局: 提前计算布局

维度五: 图片加载优化

- 图片常见问题: OOM、滑动卡顿、流量消耗大

- 优化策略:
- 图片压缩: 质量压缩、尺寸压缩
- 图片格式: WebP优于JPEG/PNG
- 三级缓存: 内存→磁盘→网络
- 按需加载: 根据控件大小采样
- 懒加载: 滚动时不加载, 停止时加载
- 主流图片加载库:
- Android: Glide、Picasso、Coil
- iOS: SDWebImage、Kingfisher
- Flutter: `cached_network_image`

性能优化方法论

- 测量先行: 不要凭感觉优化, 先量化
- 二八原则: 优化最影响体验的20%问题
- 持续监控: 性能是持续维护的, 不是一次性的
- 平衡艺术: 性能与功能、开发效率的平衡

3. 移动应用发布流程 (15分钟)

应用发布全流程

1. 功能测试 → 2. 性能测试 → 3. 安全检查 → 4. 打包签名 → 5. 提交审核 → 6. 灰度发布 → 7. 全量上线 → 8. 监控迭代

应用商店上架规范

Android 应用商店

- Google Play:
- 开发者账号: 25美元注册费
- 审核时间: 1-7天
- 政策: 内容政策、隐私政策、支付政策
- 注意: `targetSdkVersion`要求、64位支持
- 国内应用商店:
- 华为、小米、OPPO、vivo、应用宝等
- 各有独立的开发者平台与审核标准
- 需要软著、ICP备案等材料
- 审核时间: 1-3天

iOS App Store

- 开发者账号: 99美元/年 (个人)、299美元/年 (企业)

- 审核严格：
- 功能完整性
- 用户界面规范（HIG）
- 隐私政策
- 支付方式（必须使用IAP）
- 内容合规
- 审核时间：初次2-7天，更新较快
- 被拒常见原因：
- 功能不完整或有bug
- 第三方支付
- 隐私政策缺失
- 抄袭或低质量
- 误导性描述

小程序发布

- 微信小程序：
- 注册账号 → 完善信息 → 开发 → 上传 → 审核 → 发布
- 审核时间：1-7天
- 类目选择与资质要求
- 个人号 vs 企业号权限差异

版本管理策略

- 版本号规范：语义化版本（Semantic Versioning）
- 主版本号.次版本号.修订号（1.0.0）
- 主版本：不兼容的API修改
- 次版本：向下兼容的功能性新增
- 修订号：向下兼容的问题修正
- 发布周期：
- 敏捷迭代：2-4周一个版本
- 大版本：3-6个月
- 灰度发布（灰度放量）：
- 1% → 5% → 20% → 50% → 100%
- 按用户标签、设备、地区放量
- 监控核心指标，出现问题及时回滚
- 热更新：
- 目的：快速修复bug、迭代小功能
- 方案：JSBundle动态更新（React Native、Flutter）
- 注意：Apple政策限制，不能热更新原生代码

用户反馈收集与处理

- 反馈渠道：
 - 应用商店评论
 - 应用内反馈入口
 - 客服系统
 - 社群（QQ群、微信群、论坛）
- 反馈处理流程：
 - 收集 → 分类 → 优先级排序 → 排期处理 → 回复用户
- 数据驱动：
 - 崩溃分析：Bugly、Firebase Crashlytics
 - 性能监控：APM工具
 - 用户行为：埋点数据分析
 - 转化率漏斗：关键路径分析

4. 第一学时小结（5分钟）

- 回顾性能优化的五大维度与核心方法
- 梳理应用发布流程与版本管理策略
- 强调性能优化对用户体验与业务指标的重要性
- 答疑与过渡：引出下节课的代码质量与团队协作内容

第二学时：代码质量、团队协作与AI应用（45分钟）

5. 代码质量保障（12分钟）

为什么代码质量重要？

- 可维护性：降低后续开发成本
- 可靠性：减少bug与线上问题
- 可读性：团队协作效率
- 可扩展性：应对需求变化

代码规范

- 命名规范：
 - 变量、函数、类的命名原则
 - 见名知意，避免缩写
 - 遵循各语言命名惯例（驼峰、下划线、帕斯卡）
- 格式规范：
 - 缩进、空格、换行

- 代码格式化工具：Prettier、dartfmt、ClangFormat
- 注释规范：
- 为什么需要注释（解释why，不是what）
- 文档注释（API文档）
- 复杂逻辑注释
- 各平台代码规范：
- Android：Kotlin官方规范、阿里巴巴Java开发手册
- iOS：Swift API Design Guidelines、Objective-C规范
- 小程序：微信小程序开发规范
- Flutter：Effective Dart

静态分析工具

- 什么是静态分析？不运行代码，直接分析源码找问题
- 常见问题类型：
- 潜在bug：空指针、资源泄漏
- 代码风格：命名、格式
- 安全隐患：SQL注入、硬编码密钥
- 复杂度：圈复杂度、重复代码
- 主流工具：
- Android：Lint、Detekt (Kotlin) 、PMD、Checkstyle
- iOS：SwiftLint、OCLint
- JavaScript/TypeScript：ESLint、TSLint
- Flutter/Dart：dart analyze
- 最佳实践：
- IDE实时提示
- 提交前检查（pre-commit hook）
- CI/CD集成，不通过不让合并

自动化测试框架

- 测试金字塔：
- 单元测试（Unit Test）：最多、最快、最稳定
- 集成测试（Integration Test）：中等
- UI测试（UI Test/E2E）：最少、最慢、最脆弱
- 单元测试：
- 目的：验证单个函数/类的正确性
- 框架：JUnit、Mockito (Android) 、XCTest (iOS) 、flutter_test
- 覆盖率：核心业务逻辑应覆盖80%+
- 集成测试：

- 目的：验证模块间的协作
- 场景：网络请求、数据库操作、页面跳转
- UI测试：
 - 目的：模拟用户操作，验证端到端流程
- 框架：Espresso (Android) 、XCTest (iOS) 、Flutter Integration Test
- 测试驱动开发 (TDD) ：
 - 先写测试 → 再写代码 → 重构
- 优点：设计清晰、重构安全、文档作用
- 适用：核心业务逻辑、工具类

6. Git版本控制与团队协作 (10分钟)

Git团队协作基础

- 为什么需要版本控制？
- 代码备份与追溯
- 多人协作不冲突
- 功能并行开发
- 版本回滚与发布管理

分支管理策略

Git Flow (经典工作流)

- 分支类型：
 - master/main：主分支，稳定可发布
 - develop：开发分支，集成分支
 - feature/：功能分支，从develop切出
 - release/：发布分支，从develop切出
 - hotfix/*：紧急修复，从master切出
- 优点：规范严谨，适合大型项目
- 缺点：流程复杂，迭代速度慢

GitHub Flow (简化工作流)

- 核心：main分支始终可发布
- 流程：
 1. 从main切出feature分支
 2. 开发完成提交PR
 3. Code Review通过后合并到main
 4. 立即部署

- 优点：简单高效，适合敏捷开发
- 缺点：缺乏版本管理的概念

GitLab Flow (带环境分支)

- 在GitHub Flow基础上增加环境分支
- 分支：main → pre-production → production
- 优点：结合了GitHub Flow的简单与环境管理
- 适用：有明确发布周期的团队

分支策略选型建议

- 小团队/敏捷开发：GitHub Flow
- 中大型团队/版本发布制：Git Flow
- 多环境部署：GitLab Flow
- 关键原则：
 - main/master永远保持可发布
 - 功能分支短小精悍
 - 频繁合并，减少冲突

代码审查 (Code Review)

- 为什么要做Code Review?
- 发现bug与设计问题
- 统一代码风格
- 知识共享与团队成长
- 代码质量把关
- Code Review流程：
 1. 开发者提交PR/MR
 2. 自动检查 (CI: 编译、测试、静态分析)
 3. 指定Reviewer
 4. Reviewer提出意见
 5. 开发者修改回应
 6. Review通过，合并代码
- Review什么？
 - 功能正确性
 - 代码可读性与规范
 - 设计合理性
 - 性能与安全
 - 测试覆盖
- Review最佳实践：

- 小步提交，每次Review不超过400行
- 及时Review，24小时内响应
- 对事不对人，建设性意见
- 感谢Reviewer的付出

团队协作工具链

- 代码托管：GitHub、GitLab、Gitee
- 项目管理：Jira、Trello、飞书项目、钉钉项目
- 文档协作：Confluence、语雀、飞书文档
- 沟通工具：Slack、飞书、钉钉、企业微信
- CI/CD：Jenkins、GitHub Actions、GitLab CI

7. AI工具深度应用（10分钟）

AI辅助编程概览

- AI编程工具：TRAE、GitHub Copilot、Cursor、通义灵码等
- 核心能力：代码补全、代码生成、代码解释、bug修复、代码重构

场景一：代码重构

- 什么是重构？不改变外部行为，优化内部结构
- AI辅助重构的场景：
 - 重命名：更有意义的变量/函数名
 - 提取函数：大函数拆分为小函数
 - 提取类：相关字段和方法提取为类
 - 优化条件：复杂条件表达式简化
 - 设计模式：将代码重构为设计模式
 - 性能优化：找出性能瓶颈并提出优化方案
- 使用方法：
 - 选中要重构的代码
 - 描述重构目标
 - 审查AI生成的代码
 - 运行测试验证
- 注意事项：
 - 重构前确保有测试覆盖
 - 小步重构，每次只做一件事
 - 仔细审查AI生成的代码逻辑

场景二：性能优化建议分析

- AI如何辅助性能优化？

- 代码级：识别低效代码，提出优化建议
- 算法级：推荐更优算法
- 架构级：架构层面的性能优化建议
- 常见优化场景：
 - 循环优化：减少循环次数、提前退出
 - 内存优化：避免内存泄漏、对象复用
 - 数据库优化：索引优化、查询优化
 - 网络优化：请求合并、缓存策略
- 性能分析辅助：
 - 解读性能分析报告
 - 分析堆栈信息
 - 定位性能瓶颈
- 实践方法：
 1. 先用工具定位问题（Profiler、LeakCanary）
 2. 将问题代码与现象描述给AI
 3. AI分析原因并给出优化方案
 4. 评估方案可行性并实施
 5. 验证优化效果

场景三：测试用例生成

- AI生成测试用例的优势：
 - 快速生成基础测试用例
 - 覆盖边界条件
 - 生成异常场景测试
- 生成单元测试：
 - 输入：函数代码 + 功能描述
 - 输出：完整的测试用例代码
 - 覆盖：正常路径、边界条件、异常情况
- 生成UI测试：
 - 描述测试场景
 - AI生成测试脚本
 - 调整并运行验证
- 生成测试数据：
 - 构造各种类型的测试数据
 - 边界值、异常值、随机数据
- 使用建议：
 - AI生成的测试用例需要审查

- 补充业务相关的测试场景
- 测试用例也要维护更新

AI辅助开发的边界与注意事项

- AI能做什么：
 - 重复性工作
 - 样板代码生成
 - 已知问题的解决方案
 - 代码解释与学习辅助
- AI不能做什么：
 - 替代架构设计决策
 - 理解复杂业务逻辑
 - 保证代码100%正确
 - 替代测试验证
- 最佳实践：
 - 人是主导，AI是助手
 - 始终审查AI生成的代码
 - 用测试验证正确性
 - 不断学习，提升判断力

8. 课程思政：工匠精神与团队协作（8分钟）

精益求精的工匠精神

- 什么是工匠精神？

- 精益求精：追求极致，不满足于"差不多"
- 专注坚持：深耕领域，持续打磨
- 严谨细致：注重细节，不忽视小问题
- 创新进取：在传承基础上不断突破

- 在移动开发中的体现

- 性能优化：每一毫秒的改进都是价值
- 代码质量：写可读、可维护、优雅的代码
- 用户体验：关注每一个交互细节
- 产品打磨：版本迭代，持续优化

- 案例分享

- 某知名APP团队：为了启动速度提升0.1秒，优化了三个月
- 苹果公司：对细节的极致追求造就伟大产品

团队协作意识

- 为什么团队协作重要？

- 现代软件项目越来越复杂，个人英雄主义时代已过
- 1+1>2的协同效应
- 知识共享与能力互补

- 高效团队协作的要素

- 沟通：及时、清晰、有效
- 信任：互相尊重，相信队友
- 责任：对自己的代码负责，对团队负责
- 包容：尊重不同意见，接受批评
- 分享：知识共享，共同成长

- 代码审查中的协作精神

- 提交者：虚心接受意见，不抵触
- 审查者：建设性意见，对事不对人
- 共同目标：提升代码质量，一起成长

课程总结与展望

- **课程回顾：**从原生到跨平台，从小程序到鸿蒙，从基础到综合

- 技术发展趋势：

- AI辅助编程深度融合
- 跨平台技术持续演进
- 多端协同与万物互联
- 低代码/无代码兴起

- 学习建议：

- 持续学习，关注技术动态
- 动手实践，项目中成长
- 参与开源，社区中学习
- 培养软技能，全面发展

9. 第二学时小结 (5分钟)

- 总结代码质量保障的三位一体：规范、静态分析、自动化测试
- 回顾Git分支管理策略与Code Review最佳实践
- 强调AI工具是助手、人为主导的开发理念
- 全章回顾：从架构设计到性能优化，从代码质量到团队协作的完整工程实践体系
- 答疑与下节预告

五、学后测验

测验说明

本测验旨在检验学习者对性能优化、团队协作与AI应用的掌握程度，共10道题，其中单选题7道，多选题3道。

测验题目

第1题（单选）：在移动应用性能优化中，"60fps"原则的含义是什么？

- A. 每秒采样60次
- B. 每秒渲染60帧画面
- C. 每分钟60次刷新
- D. 每次请求60ms内完成

正确答案： B

答案解析： 60fps (Frames Per Second) 意味着每秒渲染60帧画面，每帧的渲染时间需要控制在约16ms ($1000\text{ms}/60$) 以内，这样用户才会觉得界面流畅。这是UI渲染性能优化的重要指标。

第2题（单选）：以下哪种情况最可能导致Android应用内存泄漏？

- A. Activity被用户手动关闭
- B. 静态单例持有Activity的Context引用
- C. 使用Application Context获取资源
- D. 在Activity的onDestroy中注销监听器

正确答案： B

答案解析： 静态变量的生命周期与应用进程一样长，如果静态单例持有Activity的Context引用，会导致Activity无法被垃圾回收，从而造成内存泄漏。应该使用ApplicationContext或者及时清除引用。

第3题（单选）：在语义化版本规范（Semantic Versioning）中，主版本号的递增表示什么？

- A. 向下兼容的bug修复
- B. 向下兼容的新功能增加
- C. 不兼容的API修改
- D. 界面样式调整

正确答案： C

答案解析： 语义化版本格式为"主版本号.次版本号.修订号"。主版本号递增表示不兼容的API修改 (Breaking Changes)，次版本号表示向下兼容的功能新增，修订号表示向下兼容的问题修复。

第4题（单选）： 在Git Flow工作流中，紧急修复bug应该从哪个分支切出？

- A. feature分支
- B. develop分支
- C. release分支
- D. master/main分支

正确答案： D

答案解析： 在Git Flow中，hotfix分支（紧急修复）必须从master/main分支切出，修复完成后合并回master和develop分支。这样可以确保线上问题快速修复，不影响正在开发的功能。

第5题（单选）： 以下关于Code Review的说法，错误的是？

- A. 可以发现bug和设计问题
- B. 有助于知识共享和团队成长
- C. 只需要关注代码格式是否规范
- D. 应该对事不对人，提出建设性意见

正确答案： C

答案解析： Code Review不仅要关注代码格式，还要关注功能正确性、设计合理性、性能安全、测试覆盖等多个方面。代码规范只是其中的一部分，更重要的是质量和设计层面的把控。

第6题（单选）： 以下哪项不是AI辅助编程的合适场景？

- A. 生成样板代码
- B. 解释不熟悉的代码
- C. 完全替代架构设计决策
- D. 辅助生成单元测试用例

正确答案： C

答案解析： AI可以辅助生成样板代码、解释代码、生成测试用例等，但不能完全替代架构设计决策。架构设计需要深刻理解业务需求、技术权衡和团队情况，这些需要人的判断力和经验。

第7题（单选）： 关于移动应用热更新，以下说法正确的是？

- A. 可以热更新任何代码，包括原生代码
- B. 热更新的目的是快速修复bug和迭代小功能
- C. Apple完全不允许任何形式的热更新
- D. 热更新可以完全替代版本发布

正确答案： B

答案解析： 热更新的主要目的是快速修复bug和迭代小功能，无需用户重新下载安装。但热更新有一定限制，Apple不允许热更新原生代码，只能更新JS/资源等，且不能改变核心功能。热更新不能完全替代版本发布。

第8题（多选）： 以下哪些是移动应用性能优化的常见维度？

- A. 启动速度优化
- B. 内存管理与泄漏检测
- C. 网络请求优化
- D. UI渲染性能调优
- E. 增加更多功能

正确答案： A、B、C、D

答案解析： 移动应用性能优化的五大核心维度是：启动速度优化、内存管理与泄漏检测、网络请求优化、UI渲染性能调优、图片加载优化。增加更多功能不属于性能优化范畴，反而可能影响性能。

第9题（多选）： 以下哪些属于静态代码分析工具能够检测的问题？

- A. 潜在的空指针异常
- B. 代码风格与命名规范问题
- C. 运行时的性能瓶颈
- D. 资源未关闭导致的泄漏
- E. 安全隐患（如硬编码密钥）

正确答案： A、B、D、E

答案解析： 静态代码分析工具通过分析源码（不运行程序）可以检测：潜在bug（如空指针、资源泄漏）、代码风格问题、安全隐患、重复代码等。运行时的性能瓶颈需要通过性能分析工具（Profiler）在运行时检测。

第10题（多选）： 关于团队协作中的Code Review，以下哪些是良好的实践？

- A. 小步提交，每次Review的代码量不宜过大
- B. Code Review只由资深工程师完成
- C. Review时对事不对人，提出建设性意见
- D. 收到Review意见后及时响应和修改
- E. Review前先通过自动化检查（编译、测试、Lint）

正确答案： A、C、D、E

答案解析： 良好的Code Review实践包括：小步提交（每次不超过400行）、对事不对人、及时响应、先过自动化检查。Code Review不应该只由资深工程师完成，团队成员都可以参与，这也是学习和成长的机会。

测验结果评估

- **10题全对：** 优秀，全面掌握性能优化与团队协作知识
- **7-9题正确：** 良好，掌握核心内容，部分细节需加强

- **5-6题正确**：及格，基本概念清楚，需要系统复习
- **5题以下**：需努力，建议重新学习本讲内容

六、课后作业与课程总结

课程总结

1. **性能优化**是移动应用的核心竞争力，需从启动、内存、网络、UI、图片多维度入手
2. **应用发布**是系统工程，从测试到上架到灰度到监控，每个环节都很重要
3. **代码质量**是团队的生命线，规范、静态分析、测试三位一体
4. **Git团队协作**需要合适的分支策略与Code Review文化
5. **AI工具**是强大的助手，但人始终是主导，需要审查与验证
6. **工匠精神与团队协作**是软件工程师必备的职业素养

课后作业

1. 选择一个你开发过的项目或常用的APP，从启动速度、内存、网络、UI四个维度进行性能分析，列出至少3个可以优化的点并提出优化方案
2. 假设你是一个5人开发团队的技术负责人，团队采用敏捷开发，2周一个迭代，请制定一套适合你们团队的Git分支管理策略与Code Review流程
3. 使用AI编程工具（如TRAIE）对你写过的一段代码进行重构，对比重构前后的差异，写一篇200字以上的反思：AI帮你做了什么？哪些地方AI做得好？哪些地方需要人工修正？
4. 结合本课程的学习，谈谈你对"工匠精神"在软件开发中的理解，以及你将如何在今后的学习和工作中践行工匠精神

课程整体回顾

- **第一章**：移动应用开发技术体系（2讲）
- **第二章**：原生开发基础（2讲）
- **第三章**：跨平台应用开发（2讲）
- **第四章**：微信小程序开发（2讲）
- **第五章**：鸿蒙多端应用开发（2讲）
- **第六章**：综合开发实践（2讲）

期末考核预告

- 考核形式：课程大作业（项目开发 + 答辩）
- 要求：
- 3-5人一组，自主选题
- 涵盖课程所学技术栈
- 完成需求分析、架构设计、编码实现、测试发布
- 提交代码、文档、演示视频
- 进行项目答辩
- 评分标准：
- 功能完整性（30%）
- 技术难度与创新性（25%）
- 代码质量与架构（20%）
- 文档与答辩（15%）
- 团队协作（10%）

七、教学反思

(课后填写)